Python scripting or automation interview will be a leetcode problem or what

It depends on the company, but **“Python scripting / automation interview”** usually means something **different from a pure LeetCode algorithm problem**.

Here are the **3 most common formats** companies use 👇

1️⃣ Practical scripting task (very common)

They give you a **small real-world automation problem** and ask you to solve it in Python.

Examples:

Example 1 – Log processing

```
You are given a log file.
Count how many times each error appears and output the top 3 errors.
```

Example 2 – File automation

```
Write a Python script that scans a folder,
finds all .txt files, and merges them into one file.
```

Example 3 – API automation

```
Call an API and extract specific data fields.
Return them as a JSON summary.
```

What they test:

- Python basics
- file handling
- string processing
- simple logic
- clean scripting

Usually **NOT heavy algorithms**.

2️⃣ Easy / Medium coding question

Sometimes they still use a **coding problem similar to LeetCode easy/medium**, but simpler.

Example:

```
Given a list of strings, group anagrams together.
```

or

```
Find duplicates in a list.
```

But normally **not dynamic programming or complex graph problems**.

3️⃣ Debug / improve a script

They may give a **broken automation script** and ask you to:

- fix bugs
- improve efficiency
- add error handling

Example:

```
This script reads a CSV but crashes on empty rows.
Fix it.
```

This is **very common in automation roles**.

✅ Typical topics you should know

Python basics

- list / dict
- loops

- functions
- comprehensions

Automation-related

- `os`
- `pathlib`
- `json`
- `csv`
- `requests`
- file reading/writing

String processing

- regex (`re`)
- parsing logs

💡 Quick rule

Interview title	Usually means
Python scripting	automation tasks
Python automation	file/API scripts
Python coding	more algorithm
Python interview (general SWE)	likely LeetCode

✅ My suggestion

Practice **both**:

- **LeetCode Easy**
- **small Python automation scripts**

💡 If you want, I can also show you **5 very common Python scripting interview questions** that companies ask (these appear **much more often than LeetCode problems**).

Provide the full Python codes about the above mentioned examples

Below are **complete Python examples** for the automation tasks I mentioned. These are the kinds of scripts commonly used in **Python scripting / automation interviews**. I kept them **clean, readable, and interview-friendly**.

1️⃣ Log Processing Script

Task: Count how many times each error appears in a log file and show the **top 3 errors**.

```
from collections import Counter

def get_top_errors(log_file):
    error_counter = Counter()

    with open(log_file, "r", encoding="utf-8") as file:
        for line in file:
            if "ERROR" in line:
                error_message = line.strip()
                error_counter[error_message] += 1

    top_errors = error_counter.most_common(3)

    return top_errors


if __name__ == "__main__":
    log_file = "system.log"

    results = get_top_errors(log_file)
```

```
    print("Top 3 Errors:")
    for error, count in results:
        print(f"{error} -> {count} times")
```

Example log file:

```
INFO server started
ERROR database connection failed
ERROR database connection failed
ERROR timeout
ERROR database connection failed
```

Output:

```
Top 3 Errors:
ERROR database connection failed -> 3 times
ERROR timeout -> 1 times
```

2️⃣ Folder Automation Script

Task: Scan a folder and merge all `.txt` files into one file.

```
import os

def merge_txt_files(folder_path, output_file):
    with open(output_file, "w", encoding="utf-8") as outfile:

        for filename in os.listdir(folder_path):
            if filename.endswith(".txt"):
                file_path = os.path.join(folder_path, filename)

                with open(file_path, "r", encoding="utf-8") as infile:
                    outfile.write(f"\n----- {filename} -----\n")
                    outfile.write(infile.read())

    print("All txt files merged successfully.")


if __name__ == "__main__":
    folder = "text_files"
    output = "merged.txt"
```

```
    merge_txt_files(folder, output)
```

Folder example:

```
text_files/
    a.txt
    b.txt
    c.txt
```

Output file:

```
----- a.txt -----
(content)

----- b.txt -----
(content)
```

3️⃣ API Automation Script

Task: Call an API and extract specific fields.

Example using the public API **jsonplaceholder**.

```
import requests

def fetch_users():
    url = "https://jsonplaceholder.typicode.com/users"

    response = requests.get(url)

    if response.status_code != 200:
        print("API request failed")
        return

    users = response.json()

    result = []

    for user in users:
        user_info = {
```

```
            "name": user["name"],
            "email": user["email"],
            "company": user["company"]["name"]
        }

        result.append(user_info)

    return result


if __name__ == "__main__":
    users = fetch_users()

    print("User Summary:")
    for user in users:
        print(user)
```

Output example:

```
{'name': 'Leanne Graham', 'email': 'leanne@example.com', 'company': 'Romag
```

4️⃣ Group Anagrams (Coding Question)

```
from collections import defaultdict

def group_anagrams(words):
    anagram_map = defaultdict(list)

    for word in words:
        key = "".join(sorted(word))
        anagram_map[key].append(word)

    return list(anagram_map.values())


if __name__ == "__main__":
    words = ["eat", "tea", "tan", "ate", "nat", "bat"]

    result = group_anagrams(words)

    print(result)
```

Output:

```
[['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
```

5️⃣ Find Duplicates in a List

```python
def find_duplicates(numbers):
    seen = set()
    duplicates = set()

    for num in numbers:
        if num in seen:
            duplicates.add(num)
        else:
            seen.add(num)

    return list(duplicates)


if __name__ == "__main__":
    nums = [1, 2, 3, 4, 2, 5, 3, 6]

    result = find_duplicates(nums)

    print("Duplicates:", result)
```

Output:

```
Duplicates: [2, 3]
```

6️⃣ CSV Debug / Fix Script

Task: Read CSV but skip empty rows safely.

```python
import csv
```

```python
def read_csv_safe(file_path):
    data = []

    with open(file_path, newline="", encoding="utf-8") as file:
        reader = csv.reader(file)

        for row in reader:
            if not row:  # skip empty rows
                continue

            data.append(row)

    return data


if __name__ == "__main__":
    rows = read_csv_safe("data.csv")

    for row in rows:
        print(row)
```

💡 Interview Tip

Automation interviews often like to see:

- ✓ clear functions
- ✓ error handling
- ✓ readable code
- ✓ use of Python libraries (`os` , `csv` , `json` , `requests`)

If you want, I can also show you **10 Python automation interview questions that appear extremely often in real interviews** (especially in **DevOps / data / automation roles**). These are **much more realistic than typical algorithm questions.**

Yes, show them with clear codes

Here are **10 very common Python scripting / automation interview tasks**, each with **clear, complete Python code**. These are closer to **real automation work** than algorithm-heavy problems.

1️⃣ Read a File and Count Word Frequency

Python

```python
from collections import Counter

def count_words(file_path):
    with open(file_path, "r", encoding="utf-8") as file:
        text = file.read().lower()

    words = text.split()
    counter = Counter(words)

    for word, count in counter.items():
        print(word, ":", count)


if __name__ == "__main__":
    count_words("sample.txt")
```

Example output

```
python : 3
automation : 2
script : 1
```

2️⃣ Find the Largest File in a Folder

Python

```python
import os

def largest_file(folder_path):
    largest = None
```

```
    largest_size = 0

    for filename in os.listdir(folder_path):
        path = os.path.join(folder_path, filename)

        if os.path.isfile(path):
            size = os.path.getsize(path)

            if size > largest_size:
                largest_size = size
                largest = filename

    return largest, largest_size


if __name__ == "__main__":
    file, size = largest_file("files")
    print("Largest file:", file)
    print("Size:", size, "bytes")
```

3️⃣ Rename Multiple Files Automatically

Example: rename

`image1.png → photo_1.png`

Python Run

```
import os

def rename_files(folder):
    count = 1

    for filename in os.listdir(folder):
        if filename.endswith(".png"):
            new_name = f"photo_{count}.png"

            old_path = os.path.join(folder, filename)
            new_path = os.path.join(folder, new_name)

            os.rename(old_path, new_path)
            count += 1
```

```
if __name__ == "__main__":
    rename_files("images")
```

4️⃣ Parse a Log File and Extract IP Addresses

Python

Run

```
import re

def extract_ips(log_file):
    pattern = r"\b\d{1,3}(?:\.\d{1,3}){3}\b"

    with open(log_file, "r") as file:
        for line in file:
            ips = re.findall(pattern, line)

            for ip in ips:
                print(ip)


if __name__ == "__main__":
    extract_ips("access.log")
```

Example output

```
192.168.1.1
10.0.0.5
```

5️⃣ Check if a Website is Online

Python

Run

```
import requests

def check_website(url):
    try:
        response = requests.get(url, timeout=5)

        if response.status_code == 200:
            print("Website is online")
        else:
```

```
            print("Website returned status:", response.status_code)

    except requests.exceptions.RequestException:
        print("Website is offline")


if __name__ == "__main__":
    check_website("https://www.google.com")
```

---

6 Convert CSV to JSON

Python Run

```
import csv
import json

def csv_to_json(csv_file, json_file):
    data = []

    with open(csv_file, newline="", encoding="utf-8") as file:
        reader = csv.DictReader(file)

        for row in reader:
            data.append(row)

    with open(json_file, "w", encoding="utf-8") as file:
        json.dump(data, file, indent=4)


if __name__ == "__main__":
    csv_to_json("data.csv", "data.json")
```

---

7 Remove Duplicate Lines From a File

Python Run

```
def remove_duplicates(input_file, output_file):
    seen = set()

    with open(input_file, "r") as infile, open(output_file, "w") as outfil

        for line in infile:
```

```
            if line not in seen:
                outfile.write(line)
                seen.add(line)


if __name__ == "__main__":
    remove_duplicates("input.txt", "clean.txt")
```

---

8 Monitor a Folder for New Files

Python Run

```
import os
import time

def monitor_folder(folder):
    seen_files = set(os.listdir(folder))

    while True:
        time.sleep(5)

        current_files = set(os.listdir(folder))
        new_files = current_files - seen_files

        for file in new_files:
            print("New file detected:", file)

        seen_files = current_files


if __name__ == "__main__":
    monitor_folder("watch_folder")
```

---

9 Download Files From a List of URLs

Python Run

```
import requests

def download_files(urls):
    for i, url in enumerate(urls):
        response = requests.get(url)
```

```
        filename = f"file_{i}.jpg"

        with open(filename, "wb") as file:
            file.write(response.content)

        print("Downloaded", filename)


if __name__ == "__main__":
    urls = [
        "https://example.com/image1.jpg",
        "https://example.com/image2.jpg"
    ]

    download_files(urls)
```

🔟 Simple Task Automation Script

Example: backup a folder.

Python Run

```
import shutil
import datetime

def backup_folder(source):
    date = datetime.datetime.now().strftime("%Y%m%d")

    backup_name = f"{source}_backup_{date}"

    shutil.copytree(source, backup_name)

    print("Backup created:", backup_name)


if __name__ == "__main__":
    backup_folder("project_folder")
```

💡 Interview Tip

In **Python scripting / automation interviews**, they want to see:

Good structure

```
def main():
    pass

if __name__ == "__main__":
    main()
```

Basic modules:

```
os
pathlib
csv
json
re
requests
shutil
datetime
```

Error handling:

```
try / except
```

Readable code.

✅ If you want, I can also show you **5 real Python automation interview questions used by big tech companies** (some even appear in companies like **Amazon, Stripe, and Bloomberg**), and I'll explain **how candidates are expected to solve them.**

Yes with clear codes

Here are **5 realistic Python scripting / automation interview questions** with **clear solutions and full code**. These are the kinds of tasks companies often give in **DevOps, data, QA automation, or backend scripting interviews**.

1️⃣ Process a Large Log File (Find Top IP Addresses)

Question

Given a web server log file, find the **top 5 IP addresses that made the most requests**.

Example log line:

```
192.168.1.1 - - [10/Oct/2025] "GET /index.html"
```

Python Solution

Python Run

```python
from collections import Counter

def top_ips(log_file, n=5):
    counter = Counter()

    with open(log_file, "r") as file:
        for line in file:
            parts = line.split()
            if parts:
                ip = parts[0]
                counter[ip] += 1

    return counter.most_common(n)


if __name__ == "__main__":
    result = top_ips("access.log")

    print("Top IP addresses:")
    for ip, count in result:
        print(ip, "->", count)
```

Output example

```
Top IP addresses:
192.168.1.1 -> 120
10.0.0.3 -> 95
172.16.0.5 -> 80
```

2️⃣ Recursively Find All Files of a Certain Type

Question

Write a script to **find all `.py` files inside a directory and its subdirectories**.

Python Solution

Python

Run

```python
import os

def find_python_files(root_folder):
    python_files = []

    for root, dirs, files in os.walk(root_folder):
        for file in files:
            if file.endswith(".py"):
                full_path = os.path.join(root, file)
                python_files.append(full_path)

    return python_files


if __name__ == "__main__":
    files = find_python_files("project")

    for f in files:
        print(f)
```

Example output

```
project/app.py
project/utils/helper.py
project/tests/test_api.py
```

3️⃣ Read JSON Data and Summarize Information

Question

Given a JSON file containing orders, calculate the **total sales per customer**.

Example JSON

```
[
 {"customer":"Alice","amount":100},
 {"customer":"Bob","amount":200},
 {"customer":"Alice","amount":150}
]
```

Python Solution

Python Run

```python
import json
from collections import defaultdict

def total_sales(json_file):
    totals = defaultdict(int)

    with open(json_file, "r") as file:
        data = json.load(file)

    for order in data:
        customer = order["customer"]
        amount = order["amount"]
        totals[customer] += amount

    return totals


if __name__ == "__main__":
    results = total_sales("orders.json")

    for customer, total in results.items():
        print(customer, ":", total)
```

Output

```
Alice : 250
Bob : 200
```

4️⃣ Retry a Failing API Request

Question

Write a script that calls an API and **retries up to 3 times if it fails**.

Python Solution

Python

Run

```python
import requests
import time

def fetch_data(url, retries=3):
    for attempt in range(retries):
        try:
            response = requests.get(url, timeout=5)

            if response.status_code == 200:
                return response.json()

        except requests.exceptions.RequestException:
            pass

        print("Retrying...")
        time.sleep(2)

    print("Failed after retries")
    return None


if __name__ == "__main__":
    data = fetch_data("https://jsonplaceholder.typicode.com/posts/1")

    print(data)
```

5️⃣ Parallel Download Script (Basic Automation)

Question

Download multiple URLs **faster using parallel threads**.

Python Solution

Python

Run

```python
import requests
from concurrent.futures import ThreadPoolExecutor

def download(url):
```

```
    response = requests.get(url)
    filename = url.split("/")[-1] or "file"

    with open(filename, "wb") as file:
        file.write(response.content)

    print("Downloaded:", filename)


def download_parallel(urls):
    with ThreadPoolExecutor(max_workers=5) as executor:
        executor.map(download, urls)


if __name__ == "__main__":
    urls = [
        "https://example.com/image1.jpg",
        "https://example.com/image2.jpg",
        "https://example.com/image3.jpg"
    ]

    download_parallel(urls)
```

⭐ What Interviewers Usually Evaluate

They look for these things more than fancy algorithms:

1️⃣ Clean structure

```
functions
main block
```

2️⃣ Correct libraries

Common scripting libraries:

```
os
pathlib
re
json
csv
requests
```

```
shutil
collections
```

3️⃣ Error handling


```
try / except
```

4️⃣ Readable code

Good variable names.

✅ Real tip from automation interviews

Most candidates fail because they **only practice algorithm problems** (like on LeetCode), but **automation interviews focus on practical scripting**.

If you'd like, I can also show you:

🚀 **The 7 Python scripting questions that appear extremely often in companies like Amazon, Stripe, and Bloomberg.**

They are **surprisingly simple but tricky**, and preparing them greatly increases the chance of passing the interview.

Yes with clear codes

Here are **7 Python scripting / automation interview questions that appear very often in real interviews**, with **clear explanations and full Python code**.

These focus on **practical scripting ability**, not complex algorithms like on LeetCode.

1️⃣ Tail a Log File (Like `tail -f`)

Question

Write a script that continuously prints **new lines added to a log file**.

Python Code

Python Run

```python
import time

def tail_file(file_path):
    with open(file_path, "r") as file:
        file.seek(0, 2)  # move to end of file

        while True:
            line = file.readline()

            if not line:
                time.sleep(1)
                continue

            print(line.strip())


if __name__ == "__main__":
    tail_file("system.log")
```

💡 Used in **monitoring systems and DevOps tools**.

2️⃣ Check Disk Usage of a Folder

Question

Calculate the **total size of a folder recursively**.

Python Code

Python Run

```python
import os

def folder_size(folder):
```

```
    total = 0

    for root, dirs, files in os.walk(folder):
        for file in files:
            path = os.path.join(root, file)
            total += os.path.getsize(path)

    return total


if __name__ == "__main__":
    size = folder_size("my_folder")
    print("Folder size:", size, "bytes")
```

3️⃣ Extract Email Addresses From a File

Question

Extract all **email addresses from a text file**.

Python Code

Python Run

```
import re

def extract_emails(file_path):
    pattern = r"[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+"

    with open(file_path, "r") as file:
        text = file.read()

    emails = re.findall(pattern, text)

    return emails


if __name__ == "__main__":
    emails = extract_emails("data.txt")

    for email in emails:
        print(email)
```

4️⃣ Schedule a Task to Run Every X Seconds

Question

Run a function **every 10 seconds**.

Python Code

Python Run

```python
import time

def task():
    print("Running scheduled task...")


def scheduler(interval):
    while True:
        task()
        time.sleep(interval)


if __name__ == "__main__":
    scheduler(10)
```

💡 Used in **automation jobs and cron-like scripts**.

5️⃣ Merge Multiple CSV Files

Question

Combine many CSV files into **one CSV file**.

Python Code

Python Run

```python
import csv
import os

def merge_csv(folder, output_file):

    with open(output_file, "w", newline="") as outfile:
        writer = None
```

```
        for filename in os.listdir(folder):
            if filename.endswith(".csv"):

                with open(os.path.join(folder, filename)) as infile:
                    reader = csv.reader(infile)

                    header = next(reader)

                    if writer is None:
                        writer = csv.writer(outfile)
                        writer.writerow(header)

                    for row in reader:
                        writer.writerow(row)


if __name__ == "__main__":
    merge_csv("csv_files", "merged.csv")
```

6️⃣ Detect Duplicate Files in a Folder

Question

Find duplicate files using **file hash**.

Python Code

Python Run

```
import os
import hashlib

def file_hash(path):
    hasher = hashlib.md5()

    with open(path, "rb") as file:
        while chunk := file.read(4096):
            hasher.update(chunk)

    return hasher.hexdigest()


def find_duplicates(folder):
    hashes = {}
```

```
    for root, dirs, files in os.walk(folder):
        for file in files:
            path = os.path.join(root, file)

            h = file_hash(path)

            if h in hashes:
                print("Duplicate:", path, "and", hashes[h])
            else:
                hashes[h] = path


if __name__ == "__main__":
    find_duplicates("my_folder")
```

7️⃣ Simple Web Scraper

Question

Extract **all links from a webpage**.

Python Code

Python Run

```
import requests
import re

def get_links(url):
    response = requests.get(url)

    html = response.text

    links = re.findall(r'href="(.*?)"', html)

    return links


if __name__ == "__main__":
    links = get_links("https://example.com")

    for link in links:
        print(link)
```

⭐ What Automation Interviewers Really Want

They evaluate:

1️⃣ File handling

```
open
read
write
```

2️⃣ System operations

```
os
pathlib
shutil
```

3️⃣ Data processing

```
json
csv
regex
```

4️⃣ Networking

```
requests
API calls
```

5️⃣ Automation logic

```
loops
timers
monitoring
```

💡 Reality of scripting interviews

Many candidates expect **hard algorithm questions**, but automation interviews focus on **practical scripting tasks rather than heavy problems from LeetCode**.

✅ If you'd like, I can also show you **a full 60-minute Python automation interview simulation** (exactly like what companies give candidates), including:

- problem statement
- input/output examples
- the expected solution
- the optimized solution

This is **extremely useful for interview preparation**.

Yes with clear codes

Below is a **realistic 60-minute Python scripting / automation interview simulation**. This type of task is common in **automation / DevOps / data roles**. It is usually **more practical than algorithm questions from LeetCode**.

🧪 Python Automation Interview Simulation

Problem

You are given a **log file** containing website access records.

Each line looks like:

```
2026-03-01 10:15:23 192.168.1.5 /login 200
2026-03-01 10:15:24 192.168.1.8 /home 200
2026-03-01 10:15:25 192.168.1.5 /cart 500
```

Each line contains

```
timestamp  ip_address  endpoint  status_code
```

🎯 Tasks

Write a Python script that:

1️⃣ Counts **total requests**
2️⃣ Finds the **top 3 IP addresses by request count**
3️⃣ Counts **how many requests returned errors** (`status >= 400`)
4️⃣ Finds the **most visited endpoint**

📂 Example Input

`access.log`

```
2026-03-01 10:15:23 192.168.1.5 /login 200
2026-03-01 10:15:24 192.168.1.8 /home 200
2026-03-01 10:15:25 192.168.1.5 /cart 500
2026-03-01 10:15:26 192.168.1.9 /login 200
2026-03-01 10:15:27 192.168.1.5 /home 404
```

🧾 Expected Output

```
Total requests: 5

Top IPs:
192.168.1.5 -> 3
192.168.1.8 -> 1
192.168.1.9 -> 1

Error requests: 2

Most visited endpoint: /login
```

✅ Full Python Solution

Python Run

```
from collections import Counter


def analyze_log(file_path):

    total_requests = 0
    error_requests = 0

    ip_counter = Counter()
    endpoint_counter = Counter()

    with open(file_path, "r") as file:

        for line in file:

            parts = line.strip().split()

            if len(parts) != 4:
                continue

            timestamp, ip, endpoint, status = parts

            total_requests += 1

            ip_counter[ip] += 1
            endpoint_counter[endpoint] += 1

            if int(status) >= 400:
                error_requests += 1

    top_ips = ip_counter.most_common(3)

    most_visited = endpoint_counter.most_common(1)[0][0]

    return total_requests, top_ips, error_requests, most_visited


def main():

    file_path = "access.log"

    total, top_ips, errors, endpoint = analyze_log(file_path)

    print("Total requests:", total)
    print()

    print("Top IPs:")
    for ip, count in top_ips:
        print(ip, "->", count)
```

```
    print()
    print("Error requests:", errors)
    print()
    print("Most visited endpoint:", endpoint)


if __name__ == "__main__":
    main()
```

⭐ What Interviewers Evaluate

They are checking whether you can:

1️⃣ Parse text files

Python Run

```
line.split()
```

2️⃣ Use Python data structures

Python Run

```
Counter
dict
list
```

3️⃣ Write clean functions

```
analyze_log()
main()
```

4️⃣ Handle simple edge cases

Example improvement:

```
if len(parts) != 4:
    continue
```

🚀 Bonus (Senior-level improvement)

A stronger candidate might also:

- support **very large log files**
- add **error handling**
- output results to **JSON**

Example improvement:

Python Run

```python
import json
```

Save results:

Python Run

```python
with open("report.json","w") as f:
    json.dump(report,f,indent=2)
```

💡 Reality of Python Scripting Interviews

Typical problems are about:

Category	Example
File processing	logs, CSV
Automation	folder scripts
API usage	fetch + parse
Data aggregation	counters
Monitoring	system scripts

Rarely complex algorithms like those on LeetCode.

✅ If you'd like, I can also show you **the 3 hardest Python automation interview questions companies like Amazon love to ask** — and most candidates fail them. They're **much trickier than they look.**

Yes with clear codes

Here are **3 tricky Python automation interview questions** that companies sometimes ask.
They look simple but **many candidates struggle with them** because they involve **efficient scripting, memory usage, and edge cases** rather than algorithm puzzles like those on LeetCode.

1️⃣ Process a Huge File Without Running Out of Memory

Problem

You have a **10GB log file**.
Count how many times each **status code** appears.

Example log line:

```
2026-03-01 10:15:23 192.168.1.5 /login 200
```

The key is **not loading the whole file into memory**.

Python Solution

Python Run

```python
from collections import Counter

def count_status_codes(file_path):
```

```
    counter = Counter()

    with open(file_path, "r") as file:
        for line in file:
            parts = line.strip().split()

            if len(parts) < 4:
                continue

            status = parts[-1]
            counter[status] += 1

    return counter


def main():

    result = count_status_codes("access.log")

    for status, count in result.items():
        print(status, "->", count)


if __name__ == "__main__":
    main()
```

Why this is good

It processes the file **line-by-line**, so memory usage stays **very small**.

2️⃣ Find the First Non-Repeating Character

Problem

Given a string, return the **first character that appears only once**.

Example

```
input:  "automation"
output: "u"
```

Python Solution

Python Run

```python
from collections import Counter

def first_unique_char(text):

    counter = Counter(text)

    for char in text:
        if counter[char] == 1:
            return char

    return None


def main():

    text = "automation"

    result = first_unique_char(text)

    print("First unique character:", result)


if __name__ == "__main__":
    main()
```

3️⃣ Implement a Rate Limiter (Automation Systems)

Problem

Allow **only 5 requests per 10 seconds** from a user.

If more requests come, **reject them**.

Python Solution

Python Run

```python
import time
from collections import deque
```

```
class RateLimiter:

    def __init__(self, limit, window):
        self.limit = limit
        self.window = window
        self.requests = deque()

    def allow_request(self):

        current_time = time.time()

        while self.requests and current_time - self.requests[0] > self.win
            self.requests.popleft()

        if len(self.requests) < self.limit:
            self.requests.append(current_time)
            return True

        return False


def main():

    limiter = RateLimiter(5, 10)

    for i in range(10):

        if limiter.allow_request():
            print("Request allowed")
        else:
            print("Request blocked")

        time.sleep(1)


if __name__ == "__main__":
    main()
```

⭐ Why These Questions Are Tricky

Interviewers want to see if you understand:

Memory efficiency

Example:

```
read line-by-line
not file.read()
```

Correct data structures

```
Counter
deque
dict
set
```

Real automation problems

Examples:

- log processing
- API rate limits
- large data handling

🚀 Extremely Common Python Libraries for Automation Interviews

Candidates are often expected to know:

```
os
pathlib
csv
json
re
requests
collections
shutil
datetime
```

✅ If you'd like, I can also show you **the 10 Python one-liners that impress interviewers the most** (very useful in scripting interviews).

Yes with clear codes

Here are **10 useful Python one-liners** that often impress interviewers in **Python scripting / automation interviews**.
They show that you know **Pythonic ways** to solve common tasks.

1️⃣ Remove Duplicates from a List

Python Run

```python
numbers = [1, 2, 2, 3, 4, 4, 5]
unique_numbers = list(set(numbers))

print(unique_numbers)
```

Output

```
[1, 2, 3, 4, 5]
```

2️⃣ Flatten a List of Lists

Python Run

```python
matrix = [[1, 2], [3, 4], [5, 6]]

flat_list = [num for row in matrix for num in row]

print(flat_list)
```

Output

```
[1, 2, 3, 4, 5, 6]
```

---

3 Swap Two Variables

Python Run

```python
a, b = 5, 10
a, b = b, a

print(a, b)
```

Output

```
10 5
```

---

4 Count Word Frequency in a Sentence

Python Run

```python
from collections import Counter

sentence = "python automation python scripting"

word_count = Counter(sentence.split())

print(word_count)
```

Output

```
Counter({'python': 2, 'automation': 1, 'scripting': 1})
```

---

5 Get File Extension

Python Run

```python
filename = "report.csv"

extension = filename.split(".")[-1]

print(extension)
```

Output

```
csv
```

---

6 Find the Most Frequent Element

Python Run

```python
from collections import Counter

numbers = [1, 3, 3, 2, 1, 3, 4]

most_common = Counter(numbers).most_common(1)[0][0]

print(most_common)
```

Output

```
3
```

---

7 Filter Even Numbers

Python Run

```python
numbers = [1, 2, 3, 4, 5, 6]

evens = [n for n in numbers if n % 2 == 0]

print(evens)
```

Output

```
[2, 4, 6]
```

---

8 Read a File into a List

Python Run

```python
lines = open("file.txt").read().splitlines()

print(lines)
```

Output example

```
['line1', 'line2', 'line3']
```

---

9 Merge Two Dictionaries

Python Run

```python
dict1 = {"a": 1, "b": 2}
dict2 = {"c": 3, "d": 4}

merged = {**dict1, **dict2}

print(merged)
```

Output

```
{'a': 1, 'b': 2, 'c': 3, 'd': 4}
```

---

10 Sort a Dictionary by Value

Python Run

```python
scores = {"Alice": 90, "Bob": 75, "Charlie": 85}

sorted_scores = dict(sorted(scores.items(), key=lambda x: x[1]))

print(sorted_scores)
```

Output

```
{'Bob': 75, 'Charlie': 85, 'Alice': 90}
```

⭐ Why Interviewers Like These

They show you understand:

Pythonic coding

Examples:

```
list comprehensions
dictionary unpacking
Counter
lambda
```

Efficient scripting

Instead of writing **10 lines**, you can solve problems in **1–2 lines**.

💡 **Interview Tip**

In Python scripting interviews, good candidates combine:

- clean functions
- readable code
- occasional Pythonic shortcuts

Not just algorithm practice on LeetCode.

✅ If you'd like, I can also show you **the 12 Python tricks that senior engineers use in automation scripts** (these appear very often in real production code).